🧭

sutra-ai-work-progress

Feature Ownership

Jatin	Krish	Shared / Already Started
• [ ] Weakness Detection Agent	• [ ] Academic Health Monitoring Agent	• [x] Dynamic Mock Test Generator — dashboard and mock exam UI prototype created
• [ ] AI Intervention Engine	• [ ] Autonomous Study Planner	
• [ ] AI Paper Evaluator	• [ ] Exam Readiness Score	
• [ ] Adaptive Exam Simulator	• [ ] Personalized Question Bank	

High-Level Feature Checklist

- [ ] Academic Health Monitoring Agent
- [ ] Weakness Detection Agent
- [ ] Autonomous Study Planner
- [ ] AI Intervention Engine
- [x] ~~Dynamic Mock Test Generator~~
- [ ] Exam Readiness Score
- [ ] AI Paper Evaluator
- [ ] Personalized Question Bank
- [ ] Adaptive Exam Simulator

Already Created / Started

- [x] ~~Clerk authentication and student onboarding~~
- [x] ~~Protected dashboard foundation~~
- [x] ~~Mock Exam dashboard section~~
- [x] ~~Dedicated mock setup page~~
- [x] ~~Step-by-step mock exam setup flow~~
- [x] ~~Fullscreen mock exam session UI~~
- [x] ~~Question navigator with answered, seen, and unseen states~~
- [x] ~~Mock timer and submit flow~~
- [x] ~~Mock cancellation flow for tab switch and fullscreen exit~~

Completed So Far

- [x] ~~Frontend app foundation is set up with Next.js App Router.~~
- [x] ~~Backend API foundation is set up with FastAPI.~~
- [x] ~~Custom `/auth` experience is in place.~~
- [x] ~~Clerk frontend authentication is installed and wired into the app.~~
- [x] ~~`/auth` supports separate Sign up and Sign in flows.~~
- [x] ~~Clerk built-in Sign in UI is integrated.~~
- [x] ~~Student onboarding wizard is implemented for the MVP path.~~
- [x] ~~Custom Clerk signup sends student onboarding metadata.~~
- [x] ~~Clerk CAPTCHA support is added for custom signup bot protection.~~
- [x] ~~Protected `/dashboard` placeholder is implemented.~~
- [x] ~~Clerk webhook stores user and student records.~~
- [x] ~~Student onboarding fields are added to the backend model and database.~~
- [x] ~~Clerk webhook is idempotent for retry and partial-user cases.~~
- [x] ~~Local database migration and failed-signup backfill were completed.~~

- [x] ~~Frontend build/typecheck and backend compile validation passed.~~

Current Implementation Context

- Project is being developed in `/home/jatin/hackathon/sutra-ai`.
- Active branch is `feature/clerk-onboarding`.
- Frontend uses **Next.js App Router**.
- Backend uses **FastAPI**.
- Frontend auth now uses Clerk through `@clerk/nextjs`.
- `/auth` now contains a mode-driven Sign up / Sign in experience.
- `/dashboard` is protected through Clerk middleware/proxy.
- Backend Clerk webhook at `/webhooks/clerk` now persists student onboarding metadata.

Work Completed So Far

- Reviewed and agreed on an integrated frontend plus backend auth/onboarding branch strategy.
- Chosen branch name: `feature/clerk-onboarding`.
- Confirmed no separate backend branch is needed because Clerk auth, onboarding, dashboard protection, and webhook persistence are one integrated feature.
- Confirmed that signup onboarding metadata can use Clerk `unsafeMetadata` for the MVP.
- Confirmed that `unsafeMetadata` should not be treated as trusted authorization data in future production logic.
- Confirmed the correct board spelling is **ICSE**, not ICSC.
- Added Clerk frontend SDK and environment configuration.
- Implemented Clerk provider, Next 16 proxy, protected dashboard, and auth page refactor.

- Implemented student onboarding wizard and custom Clerk signup metadata submission.
- Added Clerk CAPTCHA mount for custom signup bot protection.
- Extended backend student model and webhook metadata mapping.
- Added database migration for onboarding fields and applied it locally.
- Fixed webhook retry behavior so partial user creation does not block student creation.
- Backfilled the failed signup's missing student row in the local database.

Existing Frontend Features

- Next.js App Router application is present.
- `/auth` route already exists.
- `app/auth/page.tsx` currently renders the auth page component.
- `components/auth-page.tsx` currently contains the static visual auth UI.
- Existing auth UI includes a visual shell with a left testimonial panel, logo, Home button, social auth-style buttons, and an email field.

Implemented Frontend Features

- Clerk frontend SDK is installed with `@clerk/nextjs`.
- Clerk environment variables are configured in `frontend/.env.local` and documented in `frontend/.env.example`.
- App is wrapped with `ClerkProvider`.
- Root `frontend/proxy.ts` is added for Next 16 Clerk route protection.
- `/dashboard(.*)` is protected through Clerk middleware.
- `/`, `/auth`, and static assets remain public.
- `/auth` is refactored into a mode-driven flow.
- Initial `/auth` screen shows **Sign up** and **Sign in**.
- Sign in uses Clerk's built-in `<SignIn />` component.

- Sign up uses a custom onboarding wizard first, then creates a Clerk signup.
- Signup metadata is sent through Clerk `unsafeMetadata`.
- Custom signup includes Clerk CAPTCHA support through the `clerk-captcha` mount element.
- Successful signup redirects to `/dashboard`.
- Protected `/dashboard` placeholder page is added.
- Dashboard placeholder shows `Dashboard`, `Welcome to Sutra AI`, and Clerk `UserButton`.

Implemented Signup Wizard

- Step 1: Account type.
- Student is enabled.
- Institute is shown as coming soon and does not proceed.
- Step 2: Student type.
- Individual is enabled.
- Belongs to an institute is shown as coming soon and does not proceed.
- Step 3: Class.
- Supports 10th and 12th.
- Step 4: Board.
- CBSE is enabled.
- GSEB is disabled or marked coming soon.
- ICSE is disabled or marked coming soon.
- Step 5: Stream.
- Supports Science and Commerce.
- Step 6, only for Science: PCB, PCM, or PCMB.
- Commerce skips the science group step.
- After all required selections, Clerk signup is shown.

Signup Metadata Contract

```
{
  "role": "student",
  "student_type": "individual",
  "class_level": "10th | 12th",
  "board": "CBSE",
  "stream": "science | commerce",
  "science_group": "pcb | pcm | pcmb",
  "onboarding_complete": true
}
```

Existing Backend Features

- FastAPI backend exists under `/home/jatin/hackathon/sutra-ai/backend`.
- `app/main.py` is present.
- `app/routes/webhooks.py` is present.
- `app/routes/auth.py` is present with placeholder auth routes.
- `app/models/user.py` is present.
- `app/models/student.py` is present.
- `app/database.py` is present.
- `app/create_tables.py` is present.
- `app/schemas/student.py` is present.
- Clerk webhook route exists at `/webhooks/clerk`.
- Webhook currently verifies Clerk webhook requests with `CLERK_WEBHOOK_SECRET`.
- Webhook currently handles `user.created`.
- Webhook currently reads minimal metadata: `role` and `student_type`.
- Webhook currently creates `User` and `Student` rows.

- Current `Student` model includes `id`, `user_id`, `full_name`, `is_individual`, and `institute_id`.

Implemented Backend Features

- Extended the `Student` model with onboarding fields.
- Added `class_level` as nullable string.
- Added `board` as nullable string.
- Added `stream` as nullable string.
- Added `science_group` as nullable string.
- Added `onboarding_complete` as non-null boolean with default `False`.
- Added `backend/app/models/__init__.py` to register `User` and `Student` models.
- Updated Clerk webhook metadata mapping to read from `unsafe_metadata`.
- Stored onboarding metadata into the `students` table when role is `student`.
- Made webhook creation idempotent for retries.
- Fixed partial user creation behavior by creating missing student rows when a webhook retry finds an existing user.
- Left placeholder backend `/auth/login` and `/auth/register` routes untouched in this pass.

Database Migration

- Created `backend/migrations/001_add_student_onboarding_fields.sql`.
- Migration adds missing student onboarding columns using Postgres-compatible `ALTER TABLE`.
- Migration uses `ADD COLUMN IF NOT EXISTS`.
- Migration is needed because `create_all()` will not add columns to existing tables.
- Migration was applied locally after the first signup test exposed missing columns.

- The failed signup's missing student row was backfilled locally with the captured onboarding metadata.

Environment Updates

- Frontend `.env.local` contains Clerk publishable and secret keys for local testing.
- Frontend redirects point successful sign-in and signup to `/dashboard`.
- Frontend `.env.example` documents required Clerk variables.
- Backend `.env.example` includes `DATABASE_URL` and `CLERK_WEBHOOK_SECRET`.
- Backend commands should be run with the backend virtualenv activated from `backend/.venv`.

Validation Completed

- Frontend `npm run lint` passed.
- Frontend `npm run build` passed.
- Backend compile check passed with `source .venv/bin/activate && python -m compileall app`.
- Frontend dev server was started at `http://localhost:3000` for local testing.
- Signup flow reached Clerk webhook and created a new `users` row.
- Missing database columns were identified from the first signup test and fixed by applying the migration.
- Missing `students` row from the failed signup was backfilled locally.
- Specific failed signup was verified with populated onboarding fields.
- Clerk custom signup CAPTCHA issue was fixed and frontend validation was rerun successfully.

External References Used

- Clerk Next.js App Router quickstart: https://clerk.com/docs/nextjs/getting-started/quickstart

- Clerk middleware reference: https://clerk.com/docs/reference/nextjs/clerk-middleware
- Clerk custom onboarding guide: https://clerk.com/docs/guides/development/add-onboarding-flow
- Clerk custom signup bot protection guide: https://clerk.com/docs/guides/development/custom-flows/authentication/bot-sign-up-protection